
Algorithm 1 合并比例与分数表 `build_id_data_dict`

Input: 比例表; 分数表; 连接方式 `merge_how`

Output: `id_data_dict[id] = (times, ratios, scores)`

1: `merged` $\leftarrow$ 按 `merge_how` 在 `(id, time)` 上连接两张表

2: **for** `merged` 中的每个 `(id, rows)` **do**

3: 按 `time` 排序 `rows`

4: `id_data_dict[id]` $\leftarrow$ `(time, ratio, score)`

5: **end for**

6: **return** `id_data_dict`

Algorithm 2 编码一个比例 RatioFeatureEncoder

Input: 比例 x , 形状为 $(\dots, 1)$

Output: 长度为 d_{model} 的嵌入; 预训练时还有一个标量分数

1: $h \leftarrow \text{ReLU}(\text{Linear}(1 \rightarrow \text{hidden_dim})(x))$

2: $\text{embedding} \leftarrow \text{Linear}(\text{hidden_dim} \rightarrow d_{\text{model}})(h)$

3: $\text{score} \leftarrow \text{Linear}(d_{\text{model}} \rightarrow 1)(\text{embedding})$

// 仅预训练

4: **return** `get_feature` 返回 embedding; 预训练前向返回 $(\text{embedding}, \text{score})$

Algorithm 3 预训练编码器 `pretrain_ratio_encoder`

Input: 若干 (time, ratio, score) 序列; epochs、lr、batch_size

Output: 处于 eval 模式的编码器

```
1: pairs  $\leftarrow$  所有序列中的 (ratio, score)
2: for epoch  $\leftarrow$  1 to epochs do
3:   for pairs 的每个小批量 do
4:     ( $\cdot$ , predicted_score)  $\leftarrow$  encoder(ratio)
5:     用均方误差对 Adam 更新
6:   end for
7: end for
8: encoder.eval()
9: return encoder
```

// 时间不是输入

Algorithm 4 建立高斯过程数据集 `build_global_dataset`

Input: 序列; 已冻结的编码器

Output: (x, score) 数据集, x 的形状为 $(1 + d_{\text{model}})$

- 1: **for** 每个序列中的 $(\text{time}, \text{ratio}, \text{score})$ **do**
 - 2: $\text{embedding} \leftarrow \text{encoder.get_feature}(\text{ratio})$
 - 3: $x \leftarrow$ 拼接 $(\text{ratio}, \text{embedding})$
 - 4: 保存 (x, score)
 - 5: **end for**
 - 6: **return** 堆叠后的数据集
-

Algorithm 5 拟合深度核高斯过程 `train_ocrp_model`

Input: `id_data_dict`; 各层宽度; 学习率; 轮数

Output: 检查点 `OCR_P_model.pth`

- 1: 预训练编码器, 然后冻结其全部参数
 - 2: `dataset` $\leftarrow$ `build_global_dataset(encoder)`
 - 3: `(train_set, val_set)` $\leftarrow$ 按 `val_ratio` 随机划分 `dataset`
 - 4: `likelihood` $\leftarrow$ `GaussianLikelihood`
 - 5: `gp` $\leftarrow$ `DeepKernelGP(trainx, trainy)`
 - 6: `feature` $\leftarrow$ 从 $1 + d_{\text{model}}$ 到 d_{feat} 的 ReLU 多层感知机
 - 7: `mean` $\leftarrow$ 0; `kernel` $\leftarrow$ 缩放 $\times$ 每个潜在维度一个长度尺度的 RBF
 - 8: **for** `epoch` $\leftarrow$ 1 **to** `n_epochs` **do**
 - 9: `loss` $\leftarrow$ 归约 ($-\text{exact_marginal_log_likelihood}(\text{gp}(\text{train}_x), \text{train}_y)$)
 - 10: 用 Adam 更新高斯过程和似然
 - 11: **end for**
 - 12: 保存 `encoder`、`gp`、`likelihood`、`trainx`、`trainy` 和 `config`
 - 13: **return** 检查点路径
-

Algorithm 6 预测最优比例 `predict_optimal_ratio`

Input: 高斯过程; 已冻结的编码器; 区间 $[low, high]$; n_{starts} ; n_{steps}

Output: (best_ratio, best_mean)

```
1: best_mean  $\leftarrow -\infty$ 
2: for linspace(low, high,  $n_{\text{starts}}$ ) 中的每个 start do
3:   ratio  $\leftarrow$  start
4:   for step  $\leftarrow 1$  to  $n_{\text{steps}}$  do
5:      $x \leftarrow$  拼接 (ratio, encoder.get_feature(ratio))
6:     用  $-gp(x).mean$  更新 Adam
7:     把 ratio 限制在  $[low, high]$ 
8:   end for
9:   if  $gp(ratio).mean > best\_mean$  then
10:    保留这个 ratio
11:   end if
12: end for
13: return best_ratio, best_mean
```

Algorithm 7 推荐下一个比例 `ucb_acquisition`

Input: 高斯过程; 似然; 已冻结的编码器; 区间 $[\text{low}, \text{high}]$; $n_{\text{candidates}}$; β

Output: (`best_ratio`, `best_ucb`)

```
1: best_ucb  $\leftarrow -\infty$ 
2: for linspace(low, high,  $n_{\text{candidates}}$ ) 中的每个 ratio do
3:    $x \leftarrow$  拼接 (ratio, encoder.get_feature(ratio))
4:   posterior  $\leftarrow$  gp(x)
5:   ucb  $\leftarrow$  posterior.mean +  $\beta \cdot$  posterior.std
6:   if ucb > best_ucb then
7:     保留这个 ratio
8:   end if
9: end for
10: return best_ratio, best_ucb
```

Algorithm 8 拟合一个细胞群 `RatioTimePredictor.fit_id`

Input: id; times; ratios; fit_ode; model_type

Output: 保存的残差模型、动力学参数和历史

```
1: if 序列短于 min_points then
2:   return 跳过该 id
3: end if
4: if fit_ode then
5:   parameters  $\leftarrow$  对常微分方程均方误差做差分进化
6:   if 该搜索失败 then
7:     parameters  $\leftarrow$  有界局部最小化, 或默认参数
8:   end if
9: else
10:  parameters  $\leftarrow$  默认动力学参数
11: end if
12: ode_ratio  $\leftarrow$  在每个观测时间上的 BxB1 常微分方程
13: residual  $\leftarrow$  ratios - ode_ratio
14: if model_type 为 gpr then
15:   拟合从时间到残差的高斯过程
16: else
17:   拟合从时间到残差的多项式
18: end if
19: 保存残差模型、参数和 (times, ratios)
```

Algorithm 9 查询达到目标比例的时间 `RatioTimePredictor.query`

Input: id; target_ratio; max_time

Output: (found, time)

```
1: if 某条历史记录满足  $|\text{ratio} - \text{target\_ratio}| \leq \text{tolerance}$  then
2:   return 其中最早的时间
3: end if
4: if 目标高于全部观测比例, 且最后一段斜率  $\leq 0$  then
5:   return 未找到
6: end if
7: if 目标低于全部观测比例, 且最后一段斜率  $\geq 0$  then
8:   return 未找到
9: end if
10:  $t_{\text{left}} \leftarrow$  最后一次观测时间
11:  $t_{\text{right}} \leftarrow \text{max\_time} \times \text{horizon\_factor}$ 
12: for iteration  $\leftarrow 1$  to search_iter do
13:   在  $t_{\text{left}}$  到  $t_{\text{right}}$  的均匀网格上计算 predict_ratio
14:   if predicted_ratio - target_ratio 没有变号 then
15:     return 未找到
16:   end if
17:   把  $[t_{\text{left}}, t_{\text{right}}]$  缩到第一个变号区间
18:   if 区间窄于 time_atol then
19:     break
20:   end if
21: end for
22:  $t \leftarrow [t_{\text{left}}, t_{\text{right}}]$  的中点
23: if  $|\text{predict\_ratio}(t) - \text{target\_ratio}| \leq \text{query\_tol\_factor} \times \text{tolerance}$  then
24:   return (found,  $t$ )
25: end if
26: return 未找到
```
